DIPLOMADO EN CIBERSEGURIDAD — MÓDULO PRÁCTICO

# Implementación Práctica: Snyk, CLI y MCP en Cursor

Guía paso a paso para auditoría automatizada y remediación asistida en el entorno de desarrollo

---

## 🎯 Objetivo del Módulo

Dominar el ciclo completo de seguridad en el entorno de desarrollo: desde la autenticación hasta el bloqueo de vulnerabilidades en CI/CD, utilizando Snyk CLI y el protocolo MCP integrado en Cursor.

## Contenidos del Módulo

01

### Configuración del CLI de Snyk

Autenticación, instalación y comandos de escaneo SCA/SAST

02

### Integración del Protocolo MCP

Arquitectura, registro del servidor y conexión con Cursor Agent

03

### Flujos de Remediación en Vivo

Reportes automáticos, corrección de código y políticas de bloqueo

Made with GAMMA

PASO 1 DE 9

# Autenticación y Credenciales de Snyk

Antes de ejecutar cualquier escaneo, es necesario establecer una identidad verificada ante la plataforma Snyk. Este paso es el fundamento de seguridad de todo el flujo.

### Crear Cuenta

Accede a **app.snyk.io** y registra tu cuenta organizacional. Puedes usar SSO con GitHub, GitLab o Google Workspace según la configuración corporativa.

### Obtener API Token

Navega a **Account Settings → General → API Token**. Este token identifica tu sesión ante la API REST de Snyk y habilita el acceso programático a todos los endpoints.

### Guardar el Token

Cópialo y almacénalo en un gestor de secretos (Vault, 1Password, AWS Secrets Manager). Será referenciado como variable de entorno `SNYK_TOKEN` en todos los pasos siguientes.

⚠ ⚠️ **Seguridad Crítica:** El token otorga acceso completo a escaneos y métricas de la organización. Trátalo como un secreto sensible. **Nunca lo subas a Git ni lo expongas en logs públicos.** Rotar el token inmediatamente si se detecta exposición.

Made with GAMMA

PASO 2 DE 9

# Despliegue de Snyk CLI en la Terminal

## Secuencia de Instalación y Verificación

El CLI es el motor base que permite ejecutar pruebas tanto en local como en scripts de integración continua. Requiere Node.js ≥ 14 y npm instalados previamente.

```
# Instalación global vía npm
npm install -g snyk

# Autenticación automática mediante navegador
snyk auth

# Verificación de conexión y sesión activa
snyk whoami
```

El comando `snyk auth` abre el navegador por defecto y completa el flujo OAuth. Al regresar a la terminal, la sesión queda persistida en `~/.config/configstore/snyk.json`.

## Resultado Esperado

Tras ejecutar `snyk whoami`, la terminal devuelve el nombre de usuario y la organización asociada al token:

**Username: usuario@empresa.com**
**Org: mi-organizacion**
**API: https://api.snyk.io**

---

## Alternativas de Autenticación

### Variable de Entorno

`export SNYK_TOKEN="tu_token"` — recomendado para entornos CI/CD donde no hay interacción de navegador.

### Token directo

`snyk auth TU_TOKEN` — autenticación headless para scripts de automatización o Docker containers.

Made with GAMMA

PASO 3 DE 9

# Comandos de Escaneo y Análisis

El CLI de Snyk provee cuatro comandos principales organizados por tipo de análisis. Cada uno genera salida con CVSS score, ubicación de vulnerabilidad y versiones mínimas recomendadas para parchear.

## SCA — Dependencias

```
snyk test
```

Software Composition Analysis. Analiza el árbol de dependencias completo (directas e indirectas) contra la base de datos de vulnerabilidades de Snyk.

## SAST — Código Fuente

```
snyk code test
```

Static Application Security Testing. Analiza el código fuente propio en busca de patrones inseguros: inyección SQL, XSS, deserialización insegura, etc.

## Filtrado por Severidad

```
snyk test \
  --severity-threshold=high
```

Devuelve únicamente hallazgos de severidad **high** y **critical**. Ideal para fallar pipelines de CI/CD sin ruido de hallazgos menores.

## Monitor Continuo

```
snyk monitor
```

Registra una instantánea del proyecto en el panel central de Snyk. Habilita alertas automáticas cuando emergen nuevas CVEs que afecten las dependencias registradas.

**Análisis de salida:** Cada hallazgo incluye identificador CVE/CWE, puntuación CVSS v3, ruta exacta en el árbol de dependencias, versión vulnerable vs. versión mínima segura, y enlace a la advisory completa.

Made with GAMMA

PASO 4 DE 9

# Arquitectura del Protocolo MCP

## ¿Qué es el Model Context Protocol?

El **Model Context Protocol (MCP)** es un estándar abierto desarrollado por Anthropic para conectar modelos de lenguaje con herramientas locales y fuentes de datos externas de forma estructurada y segura.

---

## Ventaja Estratégica sobre Extensiones Tradicionales

A diferencia de las extensiones clásicas de IDE que solo leen archivos, el agente de IA mediante MCP adquiere **"manos"** para:

→ **Consultar activamente**

la base de datos de vulnerabilidades de Snyk en tiempo real

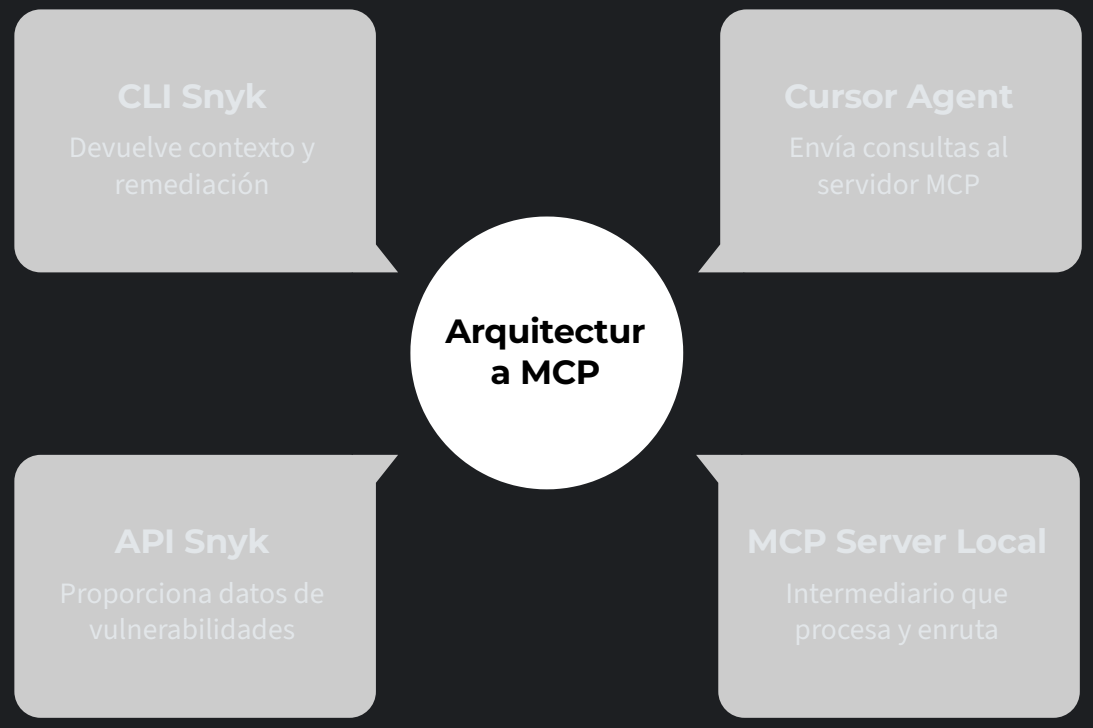
→ **Evaluar archivos**

del proyecto con contexto completo del repositorio

→ **Aplicar parches**

contextualizados de forma autónoma con revisión humana

## Flujo de Datos MCP




Cursor Agent ↔ MCP Server local ↔ Snyk API / CLI

---

## Componentes del Stack

### Cursor Agent

Motor de razonamiento LLM con capacidad de invocar herramientas MCP registradas

### @snyk/mcp-server

Proceso Node.js local que expone las capacidades de Snyk como herramientas MCP

### Snyk API / CLI

Backend de análisis con base de datos de vulnerabilidades actualizada diariamente

Made with GAMMA

PASO 5 DE 9

# Registro del Servidor MCP en Cursor

## Archivo de Configuración

Crea o edita el archivo `.cursor/mcp.json` en la raíz del proyecto (o en el directorio global de Cursor para aplicarlo a todos los proyectos):

```
{
  "mcpServers": {
    "snyk": {
      "command": "npx",
      "args": ["-y", "@snyk/mcp-server"],
      "env": {
        "SNYK_TOKEN": "TU_API_TOKEN_AQUI"
      }
    }
  }
}
```

El campo `"-y"` en args permite a npx descargar e instalar el paquete `@snyk/mcp-server` automáticamente sin confirmación interactiva, facilitando el onboarding del equipo.

## Proceso de Verificación

### 1 Guardar el archivo

Asegúrate de que el JSON sea válido. Un error de sintaxis impedirá que Cursor cargue el servidor MCP.

### 2 Abrir configuración

Navega a **Cursor Settings → Features → MCP** para ver la lista de servidores registrados.

### 3 Verificar indicador verde

El servidor `snyk` debe aparecer con estado **activo (verde)**. Un estado rojo indica error en el token o en la instalación de npx.

⚠ ⚠️ **Nunca** comitees `.cursor/mcp.json` con el token en texto plano. Usa `SNYK_TOKEN` como variable de entorno del sistema y referencia `${SNYK_TOKEN}` en el JSON, o añade el archivo a `.gitignore`.

Made with GAMMA

PASO 6 DE 9

# Interacción Agente-Herramienta con Snyk

Con el servidor MCP registrado y activo, el Cursor Agent puede invocar las capacidades de Snyk directamente desde el Composer. Este es el núcleo de la remediación asistida por IA.

## Configuración del Modo Agent

En Cursor Composer, cambia el modo de *Chat* a **Agent** mediante el selector en la esquina superior del panel. Esto habilita la ejecución de herramientas externas registradas vía MCP.

---

## Prompt de Auditoría Inicial

Utiliza el siguiente prompt estructurado para lanzar el análisis completo:

> *"Audita el proyecto utilizando las herramientas de Snyk e identifica vulnerabilidades críticas en el código y en las dependencias."*

## Comportamiento Visible del Agente

Al ejecutar el prompt, el agente realiza una secuencia observable de acciones que puedes auditar en el panel de Cursor:

### Llamada a Funciones MCP

El modelo muestra explícitamente qué herramienta MCP de Snyk está invocando (`snyk_test`, `snyk_code_test`), con qué parámetros y sobre qué directorio del proyecto.

### Recopilación de Identificadores

Extrae y agrupa los identificadores de fallo: CVE IDs, CWE categories, CVSS scores, rutas de archivo afectadas y árbol de dependencias vulnerable.

### Preparación de Contexto

Construye el contexto de remediación automática correlacionando cada hallazgo con el código fuente relevante del repositorio para proponer correcciones precisas.

Made with GAMMA

PASO 7 DE 9

# Generación Automática del Reporte de Auditoría

## Prompt Específico para Cursor

Una vez completado el análisis, usa el siguiente prompt en Cursor Composer (modo Agent) para generar la documentación formal:

> *"Genera un archivo SECURITY_AUDIT_REPORT.md documentando cada vulnerabilidad encontrada por Snyk. Incluye: Identificador CWE/CVE, nivel de severidad, archivo y línea afectada, vector de ataque según OWASP y el plan de mitigación recomendado."*

El agente creará directamente el archivo en el workspace de Cursor, listo para revisión humana y distribución a stakeholders.

## Estructura del Reporte Generado

| Sección | Contenido |
|---|---|
| Identificador | CWE-89, CVE-2024-XXXX |
| Severidad | Critical / High / Medium |
| Localización | Archivo + número de línea |
| Vector OWASP | A03: Injection, A06: Outdated Components |
| Mitigación | Versión segura + código corregido |

## Beneficios Operacionales

### Trazabilidad

Creación instantánea de evidencia para equipos de auditoría interna y externa

### Cumplimiento

Documentación automática lista para marcos como ISO 27001, SOC 2 o PCI-DSS

### Reportabilidad

Reporte estructurado listo para distribución inmediata a stakeholders sin edición manual

Made with GAMMA

PASO 8 DE 9

# Corrección del Código y Re-Escaneo

El ciclo de remediación se completa en tres fases que combinan la autonomía del agente IA con la supervisión humana obligatoria. Este balance es crítico para mantener la calidad del código.

1 2 3

## Fase 1: Solicitud de Parche

Ejecuta en Cursor Agent:

> *"Aplica la remediación recomendada por Snyk para la inyección SQL en el controlador de usuarios manteniendo la compatibilidad de tipos."*

El agente selecciona el patrón de corrección óptimo (queries parametrizadas, ORM seguro) y propone el diff.

## Fase 2: Inspección del Diff

**Revisión humana obligatoria** en Cursor de las modificaciones propuestas antes de aceptar cualquier cambio.

Validar que la corrección no introduce regresiones funcionales, que mantiene la lógica de negocio y que sigue los estándares del equipo.

## Fase 3: Comprobación de Seguridad

Re-ejecuta el análisis SAST para confirmar la extinción de la vulnerabilidad:

```
snyk code test
```

Un escaneo limpio o la ausencia del hallazgo anterior confirma la remediación exitosa. Documenta el resultado en el reporte de auditoría.

✅ **Criterio de Éxito:** La vulnerabilidad original no debe aparecer en la segunda ejecución de `snyk code test`. Si persiste, revisar que el patch fue aplicado al archivo correcto y en el scope adecuado.

Made with GAMMA

PASO 9 DE 9

# Políticas de Bloqueo y Reglas de Desarrollo

## Integración con GitHub Actions

Automatiza el bloqueo de Pull Requests que introduzcan dependencias con severidad crítica. Añade este step a tu workflow de CI/CD:

```
- name: Snyk Security Gate
  run: snyk test --severity-threshold=critical
  env:
    SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
```

Un exit code distinto de 0 bloquea el merge automáticamente. Configura la rama protegida en GitHub para requerir este check como obligatorio.

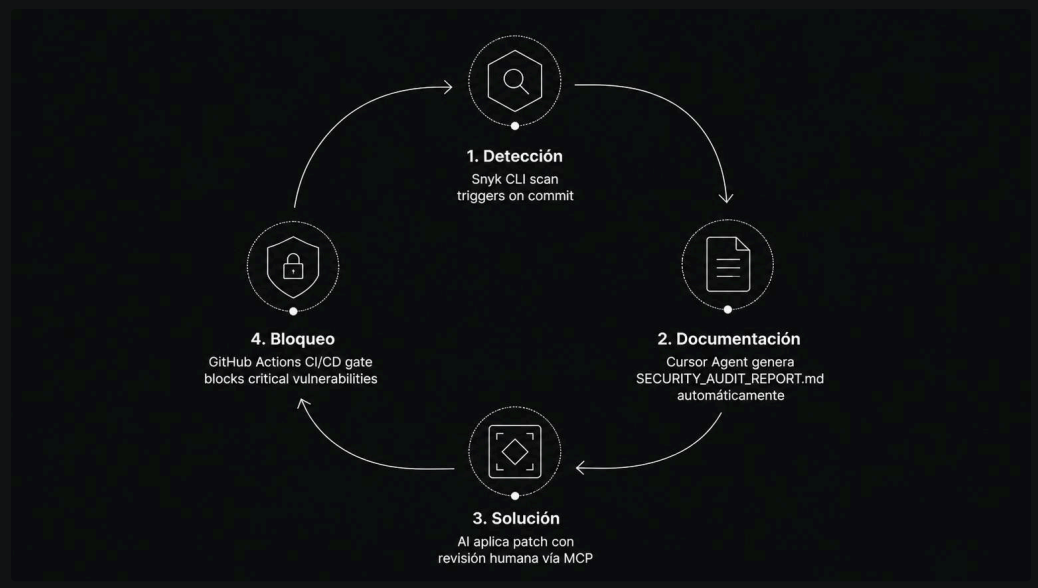
---

## Reglas para Cursor (.cursorrules)

> *"Prioriza consultas parametrizadas, esquemas de validación estricta con Zod/Joi y nunca utilices paquetes no verificados en npm o PyPI."*

Estas reglas guían al agente IA en todas las sugerencias de código, previniendo la introducción de patrones inseguros desde el origen.

## El Ciclo Cerrado de Seguridad




---

**9**

### Pasos del Flujo

Desde auth hasta CI/CD

**~0s**

### Tiempo de Reporte

Generación instantánea vía Agent

**3**

### Capas de Defensa

CLI + MCP + GitHub Actions

**Resumen Final:** Snyk + MCP + Cursor = ciclo cerrado de detección, documentación y solución en segundos. Este stack convierte la seguridad de un proceso reactivo a uno proactivo y continuo.

Made with GAMMA